

SECCIÓN 2.1

FUNDAMENTOS DE JAVASCRIPT Y NODE.JS

DESARROLLO WEB ENTORNO **SERVIDOR** 2º DAW



CIPFP AUSIÀS MARCH
CENTRE INTEGRAT PÚBLIC
DE FORMACIÓ PROFESSIONAL



¿QUÉ ES JAVASCRIPT?

Características Principales de JavaScript



- Tipado Débil (Weakly Typed): No requiere declarar tipos de datos explícitamente (string, number, boolean).
- Cambio Dinámico: Las variables pueden cambiar de tipo en tiempo de ejecución (aporta flexibilidad, pero requiere cuidado para evitar errores).
- Orientado a Objetos (OOP): Organiza la información en objetos lógicos.
- Modelos de Memoria: Distingue entre tipos de datos Primitivos y de Referencia.

VERSATILIDAD Y ENTORNOS DE EJECUCIÓN



JavaScript en todos los entornos, puntos clave:

- **Navegador (Frontend):** Entorno original del lenguaje; se encarga de escuchar eventos del usuario y manipular el DOM en tiempo real.
- **Servidor / PC (Node.js):** Permite ejecutar JavaScript fuera del navegador para crear servidores web.
- **Capacidades en Servidor:** Trabajo directo con el sistema de archivos del sistema y gestión de bases de datos.

VARIABLES Y TIPOS DE DATOS PRIMITIVOS

Declaración de Variables y Tipos Básicos

- Declaración: Uso de palabras clave como var para almacenar datos en memoria.
- Tipos de datos básicos:
- String (Texto): Valores entre comillas (ej. "Max").
- Number (Numérico): Enteros o decimales sin comillas.
- Boolean (Lógico): Valores de verdadero o falso (true / false).

Ejecución de código: Los archivos de JavaScript se ejecutan en el entorno de Node.js mediante el terminal usando el comando `node nombre_archivo.js`.

SECCIÓN 2 - EJEMPLO I

```
JS play.js ●
NODEJS-COMPLETE-GUIDE > JS play.js > ...
1  var name = 'Max';
2  var age = 29;
3  var hobbies = true;
4
5  function summarizeUser (userName, userAge, userHasHobby ){
6      return (
7          'El nombre es ' +
8          userName +
9          ' y su edad ' +
10         userAge
11     );
12 }
13 console.log(summarizeUser(name,age,hobbies));
```

LÓGICA DE FUNCIONES Y ALCANCE DE DATOS



- **Estructura:** Uso de la palabra clave function y retorno de valores con la instrucción return.
- **Funciones Puras:** Es buena práctica recibir todos los datos necesarios a través de argumentos en lugar de depender de variables externas.
- **Alcance (Scoping):** Los argumentos y variables creados dentro de una función son locales; existen únicamente dentro de ella y no se pueden acceder desde fuera.
- **Uso global vs. local:** La función se invoca pasando datos que deben estar disponibles dentro del alcance desde donde se realiza la llamada.

CUADRO RESUMEN JS

Lenguaje Débilmente Tipado

Sin asignación explícita
de tipos

Los tipos de datos se pueden
cambiar dinámicamente

Lenguaje Orientado a Objetos

Los datos se pueden
organizar en objetos lógicos

Tipos primitivos y de
referencia

Lenguaje Versátil

Se ejecuta en el navegador y
directamente en una PC/servidor

Puede realizar una amplia
variedad de tareas

DECLARACIÓN DE VARIABLES EN JAVASCRIPT MODERNO



■ **var**

- Sintaxis tradicional / obsoleta (*legacy*).
- En desuso en el desarrollo moderno de JavaScript.

■ **let**

- Se utiliza para declarar **variables cuyo valor va a cambiar** a lo largo de la ejecución.
- Sustituto directo y recomendado frente a var.

■ **const**

- Se utiliza para declarar **constantes (valores fijos)** que nunca cambian.
- Genera un error si se intenta reasignar un nuevo valor por accidente.
- **Buenas prácticas:** Usar const por defecto siempre que sea posible para dejar claras las intenciones del código y evitar modificaciones no deseadas.

EJEMPLO CORREGIDO

```
JS play.js X
NODEJS-COMPLETE-GUIDE > JS play.js > ...
4
5   const name = 'Max';
6   let age = 29;
7   const hobbies = true;
8
9   age = 30;
10  function summarizeUser (userName, userAge, userHasHobby ){
11      return (
12          'El nombre es ' +
13          userName +
14          ' y su edad ' +
15          userAge
16      );
17  }
18  console.log(summarizeUser(name,age,hobbies));
```

ARROW FUNCTIONS =>

- **¿Qué son?**
- Sintaxis moderna (ES6+) para definir funciones de forma más compacta.
- Se asignan habitualmente a una constante (`const miFuncion = (...) => { ... }`).
- Reemplazan la palabra clave `function` por una flecha (`=>`) entre los argumentos y el cuerpo.
- Sintaxis concisa: Reduce líneas de código y elimina palabras clave redundantes. Manejo del `this`: Resuelven problemas clásicos de contexto (el valor de `this` se hereda del ámbito léxico).cuerpo.

CASOS SINTAXIS

```
const summarizeUser = (userName, userAge, userHasHobby) => {  
  return (  
    'Name is ' +  
    userName +  
    ', age is ' +  
    userAge +  
    ' and the user has hobbies: ' +  
    userHasHobby  
  );  
};  
  
const add = (a, b) => {  
  return a + b;  
};  
  
console.log(add(1, 2));
```

Caso	Sintaxis	Ejemplo
Estándar	$(a, b) \Rightarrow \{ \dots \}$	$\text{const fn} = (a, b) \Rightarrow \{ \text{return } a + b; \}$
Retorno implícito	$(a, b) \Rightarrow$ expresion	$\text{const fn} = (a, b) \Rightarrow a + b$
1 solo parámetro	$\text{param} \Rightarrow \dots$	$\text{const fn} = x \Rightarrow x * 2$
0 parámetros	$() \Rightarrow \dots$	$\text{const fn} = () \Rightarrow 42$

OBJETOS Y MÉTODOS EN JAVASCRIPT I

■ Estructura básica de un objeto:

- Permiten agrupar datos (propiedades) y funciones (métodos) mediante pares clave-valor.
- Se definen entre llaves {} separando cada par con comas.
- Acceso mediante notación de punto (persona.nombre o persona.saludar()).

■ El problema de this con Arrow Functions en objetos:

- Si se usa una función flecha dentro de un objeto, this no apunta al propio objeto, sino al ámbito global (global scope), devolviendo undefined al intentar acceder a sus propiedades (e.g., this.name).

■ Sintaxis recomendada para métodos:

- Omitir los dos puntos : y la palabra clave function.
- Vincular directamente los paréntesis y el cuerpo al nombre del método para que this mantenga el contexto correcto del objeto.

OBJETOS Y MÉTODOS EN JAVASCRIPT II

Comparativa de sintaxis en métodos Incorrecto / Problemático (Arrow function):

```
1  const person = {  
2      name: 'Max',  
3      age: 29,  
4      greet: () => {  
5          console.log('Hola, soy ' + this.name);  
6      }  
7  };  
8  
9  person.greet();
```

```
1  const person = {  
2      name: 'Max',  
3      age: 29,  
4      greet () {  
5          console.log('Hola, soy ' + this.name);  
6      }  
7  };  
8  
9  person.greet();
```

ARRAYS Y MÉTODOS DE ITERACIÓN EN JAVASCRIPT



Definición y características:

- Se declaran con corchetes [].
- Permiten almacenar cualquier tipo de dato (textos, números, booleanos, objetos u otros arrays).
- Pueden contener tipos de datos heterogéneos en una misma estructura.
- Recorrido con bucle for...of: Itera directamente sobre los valores de los elementos del array de izquierda a derecha.

TRANSFORMACIÓN CON EL MÉTODO .MAP():



- **Inmutable:** Devuelve un **nuevo array** con los datos transformados, sin modificar el array original.
- Recibe una función callback que se ejecuta sobre cada elemento del array.
- Admite sintaxis concisa con *arrow functions* para retornos implícitos.

EJEMPLO TRANSFORMAR ARRAYS

```
11  const hobbies = ['Sports', 'Cooking'];
12  //for (let hobby of hobbies){
13  //    console.log (hobby);
14  //}
15
16  //hobbies.map(); transforma el array
17
18  console.log(hobbies.map(hobby => { // Arrow function
19  |     return 'Hobby: ' + hobby;
20  | })));
21
22  console.log(hobbies.map(hobby => 'Hobby: ' + hobby));
23  console.log(hobbies);
```

TIPOS POR REFERENCIA Y MUTABILIDAD EN CONST I

- Objetos y Arrays como Tipos por Referencia (Reference Types):
- La variable/constante no contiene los datos directos, sino un puntero (dirección de memoria) que apunta al lugar donde residen dichos datos.
- ¿Por qué se puede modificar un array u objeto declarado con const?
 - La restricción de const impide reasignar la dirección de memoria (el puntero).
 - Métodos mutables como `.push()` modifican el contenido en memoria al que se apunta, pero la dirección almacenada en la constante permanece idéntica.
 - No se produce ningún error porque el valor de la constante (la referencia) no ha cambiado.

TIPOS POR REFERENCIA Y MUTABILIDAD EN CONST II

```
const hobbies = ['Sports', 'Cooking'];  
hobbies.push('Programming');  
console.log(hobbies);
```

```
DE> node play2.js  
Hola, soy Max  
[ 'Sports', 'Cooking', 'Programming' ]
```

```
const hobbies = ['Sports', 'Cooking'];
```

//VÁLIDO: Modifica el contenido en memoria,
el puntero no cambia

```
hobbies.push('Programming');  
console.log(hobbies); // ['Sports', 'Cooking',  
'Programming']
```

// ERROR (TypeError): Intenta reasignar el
puntero a una nueva dirección de memoria
// hobbies = ['Programming'];

OPERADORES SPREAD Y REST (...) EN JAVASCRIPT



- **Misma sintaxis, distinto propósito:**
- Ambos utilizan tres puntos consecutivos (...).
- El nombre y comportamiento dependen exclusivamente del **lugar donde se utilicen**.

OPERADOR SPREAD (*PROPAGAR / DESESTRUCTURAR*)

- Propósito: "Extraer" o desempaquetar todos los elementos de un array o las propiedades de un objeto existente para asignarlos a uno nuevo.
- Patrón de Inmutabilidad: Permite clonar datos de forma segura sin modificar el objeto o array original (evitando errores por mutación directa).

```
29 // Copia de Array
30 const hobbies = ['Sports', 'Cooking'];
31 const hobbiesCopiados = [...hobbies, 'Programming'];
32
33 // Copia de Objeto
34 const persona = { nombre: 'Max', edad: 29 };
35 const personaCopiada = { ...persona };
```

OPERADOR REST (AGRUPAR)

- Propósito: Agrupar un número indefinido de argumentos pasados a una función dentro de un único array.
- Ubicación: Se define exclusivamente en la lista de parámetros de una función.

```
// Permite recibir N argumentos de forma flexible
const toArray = (...args) => {
  return args;
};

toArray(1, 2, 3, 4); // Retorna: [1, 2, 3, 4]
```

DESESTRUCTURACIÓN (*DESTRUCTURING*) EN JAVASCRIPT



¿Qué es?

- Sintaxis que permite "extraer" propiedades de objetos o elementos de arrays y asignarlos directamente a variables o constantes locales.
- No borra ni modifica la estructura original; solo extrae los datos necesarios y descarta el resto en ese ámbito.

I. DESESTRUCTURACIÓN DE OBJETOS ({})

- Criterio de extracción: Por nombre de propiedad (el identificador debe coincidir con la clave del objeto).
- Uso en parámetros de funciones: Permite recibir un objeto grande y extraer únicamente los campos requeridos en la firma de la función.

```
const persona = { name: 'Max', age: 29 };

// Extracción en parámetros
const printName = ({ name }) => {
  console.log(name); // 'Max'
};

// Extracción directa en declaración
const { name, age } = persona;
console.log(name, age); // 'Max' 29
```

DESESTRUCTURACIÓN DE ARRAYS ([])

- **Criterio de extracción:** Por **posición/orden** (se puede asignar cualquier nombre a las variables).

```
20 // Criterio extracción
21 const hobbies = ['Sports', 'Cooking'];
22
23 // hobby1 toma el índice 0, hobby2 el índice 1
24 const [hobby1, hobby2] = hobbies;
25
26 console.log(hobby1); // 'Sports'
27 console.log(hobby2); // 'Cooking'
```

CÓDIGO ASÍNCRONO Y PROMESAS EN JAVASCRIPT



¿Qué es el código asíncrono?

- Código que no se ejecuta ni termina de inmediato (e.g., temporizadores, consultas a bases de datos, peticiones de red).
- No bloqueante: Node.js ejecuta primero todo el código síncrono línea por línea y despacha las tareas asíncronas para resolverlas cuando finalicen.

MANEJO DE ASINCRONÍA: CALLBACKS VS. PROMESAS



Callbacks (Método tradicional):

- Funciones que se pasan como argumentos para ejecutarse cuando termina una tarea.
- **Problema (*Callback Hell*):** Si hay operaciones dependientes sucesivas, se anidan funciones dentro de funciones, volviendo el código ilegible y difícil de mantener.

Promesas (Promise) (Estándar moderno ES6):

- Objeto que representa la terminación o el fracaso eventual de una operación asíncrona.
- Estados principales: **resolve** (éxito) y **reject** (fallo/error).
- **Ventaja:** Permite el **encadenamiento lineal** mediante bloques `.then()`, eliminando el anidamiento profundo.

```

1 // 1. Definición manual de una Promesa
2 const fetchData = () => {
3   return new Promise((resolve, reject) => {
4     setTimeout(() => {
5       resolve('¡Datos recibidos!');
6     }, 1500);
7   });
8 };

```

```

// 2. Consumo y encadenamiento lineal con .then()
setTimeout(() => {
  console.log('Timer completado');

  fetchData()
    .then(text => {
      console.log(text);
      return fetchData(); // Devuelve una nueva promesa p
    })
    .then(text2 => {
      console.log(text2);
    });
}, 2000);

```

Enfoque	Estructura	Legibilidad con múltiples tareas
Callbacks	Anidamiento (fn(() => fn(() => ...)))	Mala (<i>Callback Hell</i> / Pirámide)
Promesas	Encadenamiento plano (.then().then())	Alta y estructurada

CONCLUSIÓN DEL MÓDULO DE REPASO (JAVASCRIPT MODERNO)



■ Objetivo del repaso:

- No pretende ser un curso completo de JavaScript, sino refrescar las características clave de Next-Gen JS (ES6+) indispensables para trabajar con Node.js.

■ Requisitos y preparación:

- Se requiere una base previa de JavaScript general.
- Dominar estos conceptos permite comprender con soltura todo el código backend que se desarrollará en el curso.

RESUMEN DE CONCEPTOS CLAVE VISTOS:

- Declaración: `let` (mutable) vs. `const` (inmutable por defecto).
- Funciones Flecha (Arrow Functions): Sintaxis concisa y comportamiento léxico de `this`.
- Objetos y Arrays: Tipos por referencia, mutabilidad en `const` y métodos modernos (`.map()`).
- Operadores ...: Spread (clonación/inmutabilidad) y Rest (agrupación de parámetros).
- Desestructuración: Extracción directa por clave (objetos `{}`) o posición (arrays `[]`).
- Asincronía: Código no bloqueante, callbacks y encadenamiento con Promesas (`.then()`).

ENLACES DE INTERÉS

- https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Scripting
- <https://www.w3schools.com/js/default.asp>